

Min-Heap vs. Max-Heap Master Cheat Sheet

Core Concepts, Sorting vs. Event Rules, & High-Yield DSA Problem Patterns

FAANG STAFF INTERVIEW PREP

💡 Golden Rules of Heap Selection

- **Top K / Extremes Pattern:** Reverse intuitive logic to maintain a fixed size K .
 - Find **K-th Largest** element $\rightarrow$ Use **Min-Heap of size K** (Smallest of top K sits at root).
 - Find **K-th Smallest** element $\rightarrow$ Use **Max-Heap of size K** (Largest of bottom K sits at root).
- **Intervals & Event Scheduling Pattern:** Always process items in order of start time, and track resource freeing with a **Min-Heap** (top gives the earliest ending event).
- **Greedy / Frequency Processing Pattern:** Process tasks or items dynamically using a **Max-Heap** to always execute the heaviest remaining frequency.

1. STRUCTURAL COMPARISON & MECHANICS

● MIN-HEAP

Key Property: Parent node is always $\leq$ Child nodes. Root element is always the **MINIMUM** in the structure.

JavaScript Syntax (LeetCode Runtime):

```
// Min Priority Queue
const minPQ = new MinPriorityQueue();
// Custom comparator / Object property
const minPQ = new MinPriorityQueue({
  priority: (x) => x.end // lowest end
time first
});
```

When to use:

- Need rapid access to the smallest element ($\mathcal{O}(1)$ top).
- Tracking earliest completion / availability time.
- Keeping the top K largest elements in streaming data.

● MAX-HEAP

Key Property: Parent node is always $\geq$ Child nodes. Root element is always the **MAXIMUM** in the structure.

JavaScript Syntax (LeetCode Runtime):

```
// Max Priority Queue
const maxPQ = new MaxPriorityQueue();
// Custom comparator / Object property
const maxPQ = new MaxPriorityQueue({
  priority: (x) => x.priorityVal //
highest value first
});
```

When to use:

- Need rapid access to the largest element ($\mathcal{O}(1)$ top).
- Greedily selecting highest profit or highest frequency.
- Keeping the smallest K elements in streaming data.

2. PROBLEM PATTERN CHEAT SHEET

Pattern Category	Heap Strategy	Array Sorting Required?	Classic LeetCode Examples
K-th Largest / Top K	Min-Heap (Size K)	No sorting needed. Push each element into Min-Heap; pop if size $> K$.	• LC 215: Kth Largest Element in Array • LC 703: Kth Largest Element in Stream

Pattern Category	Heap Strategy	Array Sorting Required?	Classic LeetCode Examples
K-th Smallest / Bottom K	Max-Heap (Size K)	No sorting needed. Push each element into Max-Heap; pop if size $\geq K$.	- LC 973: K Closest Points to Origin - LC 658: Find K Closest Elements
Multi-Resource Allocation (Intervals)	Min-Heap	Sort array by Start Time ($a[0] - b[0]$). Heap stores active meeting end times.	- LC 253: Meeting Rooms II - LC 1834: Single-Threaded CPU
Single Room / Max Non-Overlapping	No Heap Required	Sort array by End Time ($a[1] - b[1]$). Greedy interval comparison.	- LC 435: Non-overlapping Intervals - LC 605: Can Place Flowers
Greedy Choice + Dynamic Availability	Min-Heap (Pending) + Max-Heap (Available) OR Sorted Array + Max-Heap	Sort items by Capital / Start Time. Transfer valid choices into Max-Heap to greedily pick top profit.	- LC 502: IPO - LC 857: Minimum Cost to Hire K Workers
Frequency / Cooldown Scheduling	Max-Heap (+ Queue for Cooldown)	Populate frequency map into Max-Heap. Pop highest frequencies in blocks of $(n+1)$.	- LC 621: Task Scheduler - LC 767: Reorganize String
Two-Heap Continuous Stream	Max-Heap (Left Half) + Min-Heap (Right Half)	No prior sorting (streaming). Keep heap sizes balanced ($\Delta \leq 1$).	- LC 295: Find Median from Data Stream - LC 480: Sliding Window Median

3. KEY CODE SNIPPETS & MENTAL TRIGGERS

📌 Meeting Rooms II Logic (LC 253)

```
// 1. Sort intervals by START time
intervals.sort((a, b) => a[0] - b[0]);

// 2. Min-Heap stores end times of active rooms
const minHeap = new MinPriorityQueue();

for (const [start, end] of intervals) {
  // Free room if earliest ending meeting finishes
  if (!minHeap.isEmpty() && start >= minHeap.front().element) {
    minHeap.dequeue();
  }
  minHeap.enqueue(end);
}
return minHeap.size(); // Total rooms required
```

📌 Task Scheduler Logic (LC 621)

```
// Max-Heap processes highest remaining counts
const maxHeap = new MaxPriorityQueue();
// ...populate counts

while (!maxHeap.isEmpty()) {
  let temp = [], cycle = n + 1, count = 0;
  while (cycle > 0 && !maxHeap.isEmpty()) {
    let cnt = maxHeap.dequeue().element;
    if (cnt > 1) temp.push(cnt - 1);
    count++; cycle--;
  }
  for (let c of temp) maxHeap.enqueue(c);
  time += maxHeap.isEmpty() ? count : (n + 1);
}
```